CSC-223
Assignment 3
Work in a Pair

DEC Tokenizer

Lexical analysis, or tokenization, constitutes the first phase in the compilation process. This foundational step transforms raw source code text into a structured sequence of tokens (such as variables, operators, and literals) that represent the atomic elements of a programming language. The tokenizer acts as a compiler's front line, breaking down input text into meaningful units that subsequent compilation phases—parsing, semantic analysis, optimization, and code generation—can effectively process. In this assignment, we will implement a tokenizer for the DEC language thus enabling the symbol table to track variables and enforce visibility rules as well as facilitate parsing in our next assignment.

What You Need to Do: Continuing from the Last Assignment

You may continue to work with your partner via the same repository or a new repository for this assignment; the same repository is recommended. Please see the "Git and Github Collaboration" section in the first assignment for details.

What You Need to Do: Adding to the Project Structure

As shown in Figure 1, our Tokenizer (and tests) will be situated in the Tokenizer folder ***and*** in the Tokenizer namespace.

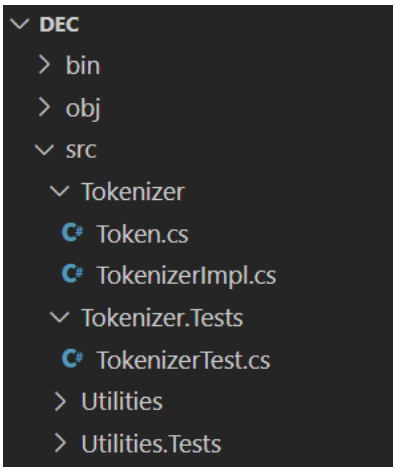

Figure 1: The desired project structure including the tokenizer.

```
namespace Tokenizer
{
    public class TokenizerImpl
    {
                ...
    }
}
```

Lexical Analysis (Tokenizing)

Lexical analysis breaks down source code into ***tokens***: the smallest meaningful units in a programming language. Think of it like identifying words and punctuation in a sentence before understanding grammar. As an example consider the statement in our DEC language: `x := (1 + 2)`. During lexical analysis, the tokenizer would transform this statement into a sequence of tokens as shown in Table 1.

Table 1: Tokenization of the DEC statement `x := (1 + 2)`

Token Value	Token Type
x	VARIABLE
:=	ASSIGNMENT
(	LEFT_PAREN
1	INTEGER
+	PLUS
2	INTEGER
)	RIGHT_PAREN

Each token has both a value (the actual text) and a type (the category it belongs to). This structured representation makes it much easier for the next phase (parsing: syntax analysis) to work with the input source code. The parser then does not need to deal with whitespace, comments, or character-by-character processing; it can focus on understanding the grammatical structure of well-defined tokens.

Tokens and Tokenizing in DEC

The tokenizer identifies and categorizes various types of tokens that form the fundamental building blocks of source code; see Table 2. These include primitive types such as variables (which must consist of lowercase alphabetic characters) and keywords like `return` (`return` is the only keyword in our language) that serve specific language functions. Integers and floating-point values allow us to define literal values that are integrated into expressions. Operators are another essential token type, divided into two main categories: (1) binary operators combining two subexpressions (i.e., addition, subtraction, etc.) and (2) the assignment operator (`:=`). The tokenizer also recognizes structural elements that define program organization: parentheses for expression grouping and curly braces for lexical scoping.

Table 2: Token types to define in the `TokenType` enumeration.

Token Catgeory	Token Types	Enumeration Type to be Implemented
Primitive Types	Variables, keywords (e.g., `return`)	VARIABLE, RETURN
Literals	(Positive) Integers and (Positive) Floats	INTEGER, FLOAT
Operators	Assignment, plus, minus, times, float division, integer division, modulus, exponentiation.	OPERATOR, ASSIGNMENT
Structure and Grouping	Parentheses, Curly braces.	LEFT_PAREN, RIGHT_PAREN, LEFT_CURLY, RIGHT_CURLY

What You Need to Do: Token.cs

A token consists of both its literal string value and an associated *type* designation. We will implement those ideas using a `Token` class. The contents of each of our files will be defined in the `Tokenizer` namespace:

```
using System;

namespace Tokenizer
{
    // enum and class definitions
}
```

The Tokenizer namespace encapsulates all token-related classes and enumerations. It provides logical organization, prevents naming conflicts with other components, and enforces a formal code-level boundary that exists regardless of physical file organization.

This file will consist of three components: a `TokenType` enumeration previously described in Table 2, a static class `TokenConstants` (Table 3), and a `Token` class.

TokenConstants. Using symbolic constants like LEFT_PAREN instead of directly using '(' improves code quality in several ways. It enhances readability by giving meaningful names to symbols, centralizes definitions for easier maintenance, and documents the purpose of each character. This approach also allows developers to easily modify the entire language syntax with a single change to the constants file—for example, switching from curly braces to square brackets as scope delimiters without modifying parser logic throughout the codebase.

A selection of the `TokenConstants` we will define is found in Table 3. Note that `TokenConstants` will be implemented as a static class in C#: it cannot be instantiated, allows direct access without object creation, and prevents inheritance of the class. The static class also signals to other developers that these values are immutable language specifications that should be accessed directly through the class name.

Table 3: A selection of token constants defined in `TokenConstants`

Constant Name	String Value	Description
PLUS	+	Addition Operator
LEFT_PAREN	(	Left parenthesis
LEFT_CURLY	{	Left curly brace
ASSIGNMENT	:=	Assignment operator
DECIMAL_POINT	.	Decimal point separator

Token. The `Token` class serves as a fundamental aggregating structure within the tokenizer, encapsulating both the lexical content (via its string value) and semantic meaning (via its `TokenType`) of individual tokens. While seemingly simple, this pairing creates a complete unit of lexical analysis that will enable us to parse with meaningful symbols rather than raw characters in the next assignment. (A `Token` object could also include location metadata such as line number, column number, and potentially character position or token length to support error reporting and debugging.)

You will need to implement the two properties (`Value` and `Type`) as well as standard methods (constructor(s), `ToString`, `Equals`, etc.). Keep it simple: `Token` should not be a complex class.

What You Need to Do: `TokenizerImpl.cs`

To break down DEC code into meaningful tokens for further processing, we will implement the `TokenizerImpl` class. At the core of this class is the `Tokenize` method, which scans through the source code character by character, creating and returning a `List<Token>` object. Implementation of the `Tokenize` method should follow a clear pattern inside of a loop that iterates character by character:

1. Check the current character,
2. Call an appropriate helper method based on that type (i.e., encountering a `:` should result in a call to a private helper method `HandleAssignment`),
3. Move forward in the string as needed, and
4. Return the corresponding token (back to the main loop within `Tokenize`).

Each private helper method encapsulates specific logic needed to properly extract and categorize each token type—whether it is distinguishing between single and multi-character operators (e.g., `/` versus `//`) or properly extracting numeric literals. At the end of each helper method, an `ArgumentException` should be raised when unexpected characters are encountered. Overall, this software design separates concerns effectively, making the code easier to debug and more maintainable and the overall algorithm easier to understand.

To ease your burden with tokenization, please review the static methods defined in the `Char` struct; in particular, `IsWhiteSpace`, `IsDigit`, and `IsLetter`.

What You Need to Do: Testing

All tests should be xUnit tests and adhere to the norms of `xUnit` including using `[Fact]` and `[Theory]`.

New to CSC-223! LLMs can be helpful tools for generating testing code for your assignments. When using an LLM to create tests, include prompts like "write thorough tests for all methods", "make sure to test with duplicate data as well" to ensure comprehensive coverage, and "use `[Theory]` and `InlineData` to avoid redundancy in testing code". ***However, be warned that generated code isn't always perfect—your professor has encountered several cases where tests were written incorrectly for the corresponding code.*** Always review each test carefully to verify it is testing what you intend and functioning properly with your implementation. It is also fair to ask the LLM to see if any cases have been missed in the tests that it generates. In fact, it is advised that you ask an LLM to 'check its work'; do not accept a first answer.

Even though you may use an LLM for testing, you will explicitly be asked about test coverage and how you know all tokens have been handled.

What You Need to Do: Commenting

Comment code consistent with the standards stated in the first assignment: when needed, use an LLM to generate comments then refine as needed.

Submitting: Source Code

You will demonstrate your source code to the instructor, in person. Be ready to demonstrate successful execution of all unit tests and describe all tests and code you were required to implement.